

Detailed Line-by-Line Explanation of DL-3 Final Notebook

This document explains the uploaded Jupyter Notebook: **DL-3 Final Notebook**. Every important code line and markdown section is explained in detail. The notebook may contain: Deep Learning concepts TensorFlow/Keras implementation Data preprocessing Training and evaluation Prediction and visualization

Cell 1 (CODE)

Original Content:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split
```

Line-by-Line Explanation:

Line	Code	Explanation
1	import pandas as pd	Imports required Python libraries or modules.
2	import matplotlib.pyplot as plt	Imports required Python libraries or modules.
3	import numpy as np	Imports required Python libraries or modules.
4	from sklearn.model_selection import train_test_split	Imports required Python libraries or modules.

Cell 2 (MARKDOWN)

Original Content:

```
### Loading the dataset
```

Line-by-Line Explanation:

Line	Code	Explanation
1	### Loading the dataset	Markdown text describing concepts or notebook sections.

Cell 3 (CODE)

Original Content:

```
df = pd.read_csv('Google Stock Prices Dataset.csv')

train_df, test_df = train_test_split(df, test_size=0.2, shuffle=False)
```

Line-by-Line Explanation:

Line	Code	Explanation
2	df = pd.read_csv('Google Stock Prices Dataset.csv')	Handles tabular data using Pandas.
4	train_df, test_df = train_test_split(df, test_size=0.2, shuffle=False)	Assigns values to variables or stores results.

Cell 4 (MARKDOWN)

Original Content:

```
#### Choosing column 'open' for prediction
```

Line-by-Line Explanation:

Line	Code	Explanation
1	#### Choosing column 'open' for prediction	Markdown text describing concepts or notebook sections.

Cell 5 (CODE)

Original Content:

```
train = train_df.loc[:, ["Open"]].values
train.shape
```

Line-by-Line Explanation:

Line	Code	Explanation
1	train = train_df.loc[:, ["Open"]].values	Assigns values to variables or stores results.
2	train.shape	General statement or notebook content.

Cell 6 (MARKDOWN)

Original Content:

```
### Feature Scaling
```

Line-by-Line Explanation:

Line	Code	Explanation
1	### Feature Scaling	Markdown text describing concepts or notebook sections.

Cell 7 (CODE)

Original Content:

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
```

Line-by-Line Explanation:

Line	Code	Explanation
1	from sklearn.preprocessing import MinMaxScaler	Imports required Python libraries or modules.
2	scaler = MinMaxScaler()	Assigns values to variables or stores results.

Cell 8 (CODE)

Original Content:

```
train_scaled = scaler.fit_transform(train)
```

Line-by-Line Explanation:

Line	Code	Explanation
1	train_scaled = scaler.fit_transform(train)	Assigns values to variables or stores results.

Cell 9 (CODE)

Original Content:

```
plt.plot(train_scaled)
plt.ylabel("Standardized Values")
plt.xlabel("Time->")
plt.show()
```

Line-by-Line Explanation:

Line	Code	Explanation
1	plt.plot(train_scaled)	Creates visualizations and graphs.
2	plt.ylabel("Standardized Values")	Creates visualizations and graphs.
3	plt.xlabel("Time->")	Creates visualizations and graphs.
4	plt.show()	Creates visualizations and graphs.

Cell 10 (CODE)

Original Content:

```
x_train = []
y_train = []
time = 60
for i in range(60,train_scaled.shape[0]):
    x_train.append(train_scaled[i-60:i,0])
    y_train.append(train_scaled[i,0])
x_train = np.array(x_train)
y_train = np.array(y_train)
```

Line-by-Line Explanation:

Line	Code	Explanation
1	x_train = []	Assigns values to variables or stores results.
2	y_train = []	Assigns values to variables or stores results.
3	time = 60	Assigns values to variables or stores results.
4	for i in range(60,train_scaled.shape[0]):	General statement or notebook content.
5	x_train.append(train_scaled[i-60:i,0])	General statement or notebook content.
6	y_train.append(train_scaled[i,0])	General statement or notebook content.
7	x_train = np.array(x_train)	Performs numerical operations using NumPy.
8	y_train = np.array(y_train)	Performs numerical operations using NumPy.

Cell 11 (CODE)

Original Content:

```
x_train.shape,y_train.shape
```

Line-by-Line Explanation:

Line	Code	Explanation
------	------	-------------

1	x_train.shape,y_train.shape	General statement or notebook content.
---	-----------------------------	--

Cell 12 (CODE)

Original Content:

```
x_train = np.reshape(x_train,newshape=(x_train.shape[0],x_train.shape[1],1))
x_train.shape
```

Line-by-Line Explanation:

Line	Code	Explanation
1	x_train = np.reshape(x_train,newshape=(x_train.shape[0],x_train.shape[1],1))	Reshape operation using NumPy.
2	x_train.shape	General statement or notebook content.

Cell 13 (MARKDOWN)

Original Content:

```
### Build model
```

Line-by-Line Explanation:

Line	Code	Explanation
1	### Build model	Markdown text describing concepts or notebook sections.

Cell 14 (CODE)

Original Content:

```
from keras.models import Sequential
from keras.layers import Dense, SimpleRNN,Dropout
```

Line-by-Line Explanation:

Line	Code	Explanation
1	from keras.models import Sequential	Imports required Python libraries or modules.
2	from keras.layers import Dense, SimpleRNN,Dropout	Imports required Python libraries or modules.

Cell 15 (CODE)

Original Content:

```
model = Sequential()

model.add(SimpleRNN(units=50,activation = "tanh", return_sequences = True,
                    input_shape = (x_train.shape[1], 1)))
model.add(Dropout(0.2))

model.add(SimpleRNN(units=50,activation = "tanh", return_sequences = True))
model.add(Dropout(0.2))

model.add(SimpleRNN(units=50,activation = "tanh", return_sequences = True))
model.add(Dropout(0.2))

model.add(SimpleRNN(units=50))
model.add(Dropout(0.2))

model.add(Dense(units=1))

model.compile(optimizer='adam', loss='mse')
model.summary()
```

Line-by-Line Explanation:

Line	Code	Explanation
1	model = Sequential()	Creates a Sequential neural network model.
3	model.add(SimpleRNN(units=50,activation = "tanh", return_sequences = True,	Assigns values to variables or stores results.
4	input_shape = (x_train.shape[1], 1)))	Assigns values to variables or stores results.
5	model.add(Dropout(0.2))	Randomly disables neurons during training to reduce overfitting.
7	model.add(SimpleRNN(units=50,activation = "tanh", return_sequences = True))	Assigns values to variables or stores results.
8	model.add(Dropout(0.2))	Randomly disables neurons during training to reduce overfitting.
10	model.add(SimpleRNN(units=50,activation = "tanh", return_sequences = True))	Assigns values to variables or stores results.
11	model.add(Dropout(0.2))	Randomly disables neurons during training to reduce overfitting.
13	model.add(SimpleRNN(units=50))	Assigns values to variables or stores results.
14	model.add(Dropout(0.2))	Randomly disables neurons during training to reduce overfitting.
16	model.add(Dense(units=1))	Adds a fully connected neural network layer.
18	model.compile(optimizer='adam',loss='mse')	Configures optimizer, loss function, and evaluation metrics.
19	model.summary()	General statement or notebook content.

Cell 16 (CODE)

Original Content:

```
model.fit(x_train,y_train,epochs=100,batch_size=30,validation_split=0.05)
```

Line-by-Line Explanation:

Line	Code	Explanation
1	model.fit(x_train,y_train,epochs=100,batch_size=30,validation_split=0.05)	Trains the deep learning model on the dataset.

Cell 17 (MARKDOWN)

Original Content:

```
### Prepare test dataset
```

Line-by-Line Explanation:

Line	Code	Explanation
1	### Prepare test dataset	Markdown text describing concepts or notebook sections.

Cell 18 (CODE)

Original Content:

```
data = pd.concat((train_df['Open'],test_df['Open']),axis=0)
```

Line-by-Line Explanation:

Line	Code	Explanation
1	data = pd.concat((train_df['Open'],test_df['Open']),axis=0)	Handles tabular data using Pandas.

Cell 19 (CODE)

Original Content:

```
test_input = data.iloc[len(data) - len(test_df) - time : ].values  
test_input.shape
```

Line-by-Line Explanation:

Line	Code	Explanation
1	test_input = data.iloc[len(data) - len(test_df) - time :].values	Assigns values to variables or stores results.
2	test_input.shape	General statement or notebook content.

Cell 20 (CODE)

Original Content:

```
test_input = test_input.reshape(-1,1)
test_input.shape
```

Line-by-Line Explanation:

Line	Code	Explanation
1	test_input = test_input.reshape(-1,1)	Assigns values to variables or stores results.
2	test_input.shape	General statement or notebook content.

Cell 21 (CODE)

Original Content:

```
test_scaled = scaler.transform(test_input)
```

Line-by-Line Explanation:

Line	Code	Explanation
1	test_scaled = scaler.transform(test_input)	Assigns values to variables or stores results.

Cell 22 (MARKDOWN)

Original Content:

```
#### Create test data set
```

Line-by-Line Explanation:

Line	Code	Explanation
1	#### Create test data set	Markdown text describing concepts or notebook sections.

Cell 23 (CODE)

Original Content:

```
x_test = []
for i in range(time,test_scaled.shape[0]):
    x_test.append(test_scaled[i - time: i,0 ])
x_test = np.array(x_test)
x_test.shape
```

Line-by-Line Explanation:

Line	Code	Explanation
1	x_test = []	Assigns values to variables or stores results.
2	for i in range(time,test_scaled.shape[0]):	General statement or notebook content.
3	x_test.append(test_scaled[i - time: i,0])	General statement or notebook content.
4	x_test = np.array(x_test)	Performs numerical operations using NumPy.
5	x_test.shape	General statement or notebook content.

Cell 24 (CODE)

Original Content:

```
x_test = np.reshape(x_test,newshape=(x_test.shape[0],x_test.shape[1],1))
x_test.shape
```

Line-by-Line Explanation:

Line	Code	Explanation
1	x_test = np.reshape(x_test,newshape=(x_test.shape[0],x_test.shape[1],1))	Performs numerical operations using NumPy.
2	x_test.shape	General statement or notebook content.

Cell 25 (CODE)

Original Content:

```
y_test = test_df.loc[:, "Open"].values
```

Line-by-Line Explanation:

Line	Code	Explanation
1	y_test = test_df.loc[:, "Open"].values	Assigns values to variables or stores results.

Cell 26 (MARKDOWN)

Original Content:

```
### Model Prediction
```

Line-by-Line Explanation:

Line	Code	Explanation
1	### Model Prediction	Markdown text describing concepts or notebook sections.

Cell 27 (CODE)

Original Content:

```
y_pred = model.predict(x_test)
```

Line-by-Line Explanation:

Line	Code	Explanation
1	y_pred = model.predict(x_test)	Uses trained model to make predictions.

Cell 28 (CODE)

Original Content:

```
y_pred = scaler.inverse_transform(y_pred)
```

Line-by-Line Explanation:

Line	Code	Explanation
1	y_pred = scaler.inverse_transform(y_pred)	Assigns values to variables or stores results.

Cell 29 (CODE)

Original Content:

```
output = model.evaluate(x=x_test,y=y_test)
```

Line-by-Line Explanation:

Line	Code	Explanation
1	output = model.evaluate(x=x_test,y=y_test)	Measures model performance on testing data.

Cell 30 (CODE)

Original Content:

```
plt.plot(y_test, color = 'red', label = 'Real price')
plt.plot(y_pred, color = 'blue', label = 'Predicted price')

plt.title('Google Stock price prediction')
plt.xlabel('Time')
plt.ylabel('Price')
plt.legend()
plt.show()
```

Line-by-Line Explanation:

Line	Code	Explanation
1	plt.plot(y_test, color = 'red', label = 'Real price')	Creates visualizations and graphs.
2	plt.plot(y_pred, color = 'blue', label = 'Predicted price')	Creates visualizations and graphs.
4	plt.title('Google Stock price prediction')	Uses trained model to make predictions.
5	plt.xlabel('Time')	Creates visualizations and graphs.
6	plt.ylabel('Price')	Creates visualizations and graphs.
7	plt.legend()	Creates visualizations and graphs.
8	plt.show()	Creates visualizations and graphs.

Overall Summary:

This notebook demonstrates important Deep Learning workflows including: Dataset preparation Model building Training neural networks Performance evaluation Prediction generation It helps understand practical implementation of deep learning using Python.